URBAN NOISE PATTERN ANALYSIS VIA MOBILE RECORDINGS

A INTERNSHIP REPORT

Submitted by

Mohammad Amaan

B. Tech - CSE (DSAI-A), 3rd Year

ROLL NO. 2301887054

ENROLLMENT NO. 2300101431

SESSION: 2025-26

In partial fulfillment for the award of the Degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
INTEGRAL UNIVERSITY
LUCKNOW, U.P.

JUNE, 2025

CERTIFICATE

This is to certify that this report entitled **URBAN NOISE PATTERN ANALYSIS VIA MOBILE RECORDINGS** " is a bonafide record of Internship presented by **Mohammad Amaan** (Reg. No- 2301887054) towards the partial fulfillment for the requirement for the award of the Degree of ***Bachelor of Technology*** in **Computer Science &** ***Engineering***, Integral University, Lucknow during the year 2025-2026.

Mr. Sankalp Agnihothri
(Internship Co-ordinator)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who contributed to the successful completion of our summer training project . First and foremost, I would like to thank **Mr. Sankalp Agnihothri**, our project supervisor, for his invaluable guidance, support, and encouragement throughout the duration of this project. His expertise and insights were instrumental in shaping the direction of our work.

I extend my heartfelt thanks to **Integral University, Lucknow** for providing us with the opportunity and resources to undertake this training. The experience and knowledge gained during this period have been immensely enriching and have significantly contributed to our professional growth.

Lastly, I would also like to acknowledge the support and assistance provided by the staff and faculty members of Computer Science & Engineering at **Integral University, Lucknow**. Their cooperation and readiness to help facilitated a smooth and productive training experience.

Thank you all for your support and contributions.

Sincerely,

Mohammad Amaan

Student's Signature

ABSTRACT

This project addresses the rising concern of noise pollution in urban environments by analyzing large-scale mobile-recorded audio data. The dataset comprises over 57,000 annotated 10-second audio clips from various boroughs of New York City, capturing events such as engine noises, sirens, construction sounds, human voices, and music. The objective was to explore how these noise events vary spatially and temporally and identify dominant sound patterns contributing to urban noise.

Using Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn, we performed exploratory data analysis, plotted temporal histograms, spatial scatter plots, pie charts, and generated a correlation heatmap. This enabled us to identify the most frequent sound sources, their co-occurrence patterns, and how noise levels differ by location and time (hour, day, week, year).

The results showed that engine sounds and human voices were among the most common, with sirens and construction-related noise prevalent during working hours. Strong correlations were observed between specific sound categories, indicating overlapping urban activities.

The conclusions drawn from this project provide valuable insights for city planners and environmental researchers. The study highlights how citizen-contributed audio and data analytics can support smarter urban planning and noise management strategies.

CONTENTS

LIST OF SYMBOLS

Symbol	Meaning
μ	Micro (used in measurement units like microseconds or dB levels)
λ	Wavelength (in theoretical signal analysis)
σ	Standard deviation (used in distribution of sound levels)
ρ	Correlation coefficient (in heatmaps and feature analysis)
t	Time (temporal variable: hour, day, week)
f	Frequency (of a sound signal)
x, y	Coordinates (used in plotting longitude and latitude)

LIST OF ABBREVIATIONS

Abbreviations	Expansions
EDA	Exploratory Data Analysis
CSV	Comma-Separated Values
ID	Identifier
NYC	New York City
ML	Machine Learning
GPS	Global Positioning System
AM	Ante Meridiem (Before Noon)
PM	Post Meridiem (After Noon)

LIST OF FIGURES

1. INTRODUCTION

1.1 Motivation and Overview

With rapid urbanization and population growth, noise pollution has emerged as a significant environmental and public health concern in major cities worldwide. Continuous exposure to high levels of urban noise—caused by traffic, construction, industrial activity, and public gatherings—has been linked to adverse effects such as stress, sleep disruption, hearing loss, and decreased productivity.

Traditional noise monitoring methods rely on stationary sensors and governmental surveys, which often lack spatial coverage and real-time feedback. However, the growing availability of mobile devices and citizen-contributed data has introduced new opportunities for participatory noise sensing.

This project is motivated by the need to utilize such participatory data to better understand **urban noise patterns** across different boroughs, locations, and time intervals. By analyzing a rich dataset of labeled 10-second audio clips collected through mobile recordings in New York City, the study aims to uncover patterns in noise events—such as engine sounds, sirens, human voices, and music—and explore how they vary **spatially** and **temporally**.

The overarching goal is to generate actionable insights that can assist in **urban planning**, **noise control policies**, and public awareness. Through data-driven analysis and visualizations, the project highlights how modern data science tools can be effectively employed to address real-world environmental issues.

1.2 Literature Survey

Urban noise monitoring has been a growing area of research due to its implications for public health, environmental quality, and city planning. Traditional approaches often rely on fixed acoustic sensor networks and periodic manual surveys. While effective in certain contexts, these systems are expensive, offer limited spatial coverage, and do not scale easily in real time.

Recent studies have shifted toward **crowdsourced and participatory sensing** methods. Projects like **SONYC (Sounds of New York City)** introduced large-scale urban acoustic monitoring by deploying sensors across New York City and leveraging machine learning to classify sound types such as jackhammers, sirens, and construction noise.

Other notable research includes the use of **mobile phone microphones** and mobile applications for noise logging and real-time reporting. These methods allow for more **dynamic data collection**, enabling researchers to analyze fine-grained variations in noise exposure across different times and locations.

Additionally, many researchers have applied **machine learning techniques** to urban sound classification, using datasets like **UrbanSound8K** and **AudioSet**, which help train models to distinguish between various environmental and anthropogenic sound events.

Visualization tools such as **heatmaps**, **correlation matrices**, and **time-series plots** have proven effective in identifying noise trends and peak activity zones, aiding urban planners and policy makers in crafting data-driven solutions.

This project builds upon these approaches by combining spatial-temporal analysis with annotated sound event data collected via mobile recordings, offering a unique blend of **citizen sensing** and **data analytics** to address the noise pollution challenge in metropolitan areas.

2. MATERIALS AND METHODOLOGY

2.1 Dataset Description

The dataset used in this project is derived from a publicly available urban sound dataset collected in New York City. It includes approximately **57,000 audio clips**, each 10 seconds long, recorded via mobile sensors deployed across different boroughs. Each audio clip is annotated with detailed metadata such as:

- Location (latitude and longitude)
- Time (hour, day, week, year)
- Sound events (e.g., engine sounds, sirens, human voices, dogs barking, music)
- Presence and proximity indicators (values: -1 = not labeled, 0 = not present, 1 = present)
- This rich annotation enabled both **spatial** and **temporal** analysis of noise patterns.

2.2 Tools and Libraries Used

The project was developed in **Python**, leveraging the following libraries:

- pandas – data loading and cleaning
- numpy – numerical operations
- matplotlib and seaborn – data visualization
- sklearn – correlation analysis and data preprocessing

All analyses and visualizations were done using a **Jupyter Notebook**.

2.3 Algorithms and Techniques

While no predictive machine learning model was trained, the project used the following **data analysis techniques**:

- **Grouping and Aggregation** to summarize trends by hour, day, or location.
- **Heatmap correlation matrix** to identify relationships among noise types.
- **Pie charts** and **bar plots** to visualize sound type distribution.
- **Histograms** for temporal activity patterns (e.g., noise by hour).
- **Scatter plots** for spatial mapping using longitude and latitude.

2.4 Program Development Workflow

1. **Data Loading and Cleaning**
   - Removed null or invalid records.
   - Standardized column names and formats.
2. **Exploratory Data Analysis (EDA)**
   - Extracted key trends based on time and location.
   - Identified most common sound events.
3. **Visualization and Plotting**
   - Generated multiple visualizations such as:
     - Temporal histograms
     - Spatial scatter plots
     - Pie charts by event type
     - Correlation heatmaps
4. **Insights Extraction**
   - Interpreted patterns to determine which sound types co-occur and when noise peaks occur.

3. FEASIBILITY STUDY

Technical

3.1. Dataset Requirements:

The successful implementation of urban noise analysis depends on a well-structured and comprehensive dataset. The following requirements were met by the dataset used in this project. The dataset used in this project is derived from a publicly available urban sound dataset collected in New York City. It includes approximately **57,000 audio clips**, each 10 seconds long, recorded via mobile sensors deployed across different boroughs

```
df= pd.read_csv('annotations.csv',low_memory=False)
print("Read file successfully")
```

3.2. Tools and Platforms:

Software Requirements: Python (with libraries like numpy, pandas), Integrated Development Environment (IDE) such as VS Code, or Jupyter Notebook

Hardware Requirements: CPU/GPU: Depending on the size of the dataset and complexity of the model, GPUs (Graphics Processing Units) can significantly accelerate training times for deep learning models.

Memory: Adequate RAM to handle large datasets and model computations.

Scalability:

Handling Growth: The system should be scalable to handle growing data volumes and more complex queries.

Performance Tuning: Regular performance tuning and optimization might be necessary.

Time

Model Validation: Testing and validating the results can take additional time to ensure the rules are meaningful and actionable.

Cost:

Software and Tools:

Open Source: Many open-source tools and libraries (streamlit, pillow (PIL), scikit libraries) can be used at no cost.

Hardware:

Computational Resources: The cost of computational resources can vary. Larger datasets might require more powerful hardware or cloud computing services (like AWS, Azure) which can increase costs.

Maintenance:

Ongoing Costs: No such cost requirements for personal level usage.

OBJECTIVE

Formulate a python project in a development environment like google colab which provide us visual prediction of future stock market turnarounds and unconditional crux using the provided data in the form of .csv

3.3 METHODOLOGY

Data Collection

The dataset utilized in this project is sourced from the SONYC Urban Sound dataset, a real-world collection of over 57,000 mobile-recorded audio clips from New York City. Each clip is 10 seconds long and is accompanied by rich metadata, including both spatial and temporal attributes.

Procedure

Importing Data Set: The dataset we will use here to perform the analysis and generate insights from the urban noise pattern data. We will use SONYC Urban Sound dataset, a real-world collection of over 57,000 mobile-recorded audio clip**s** from New York City over a time period of 4 years.

Exploratory Data Analysis: EDA is an approach to analyzing the data using visual techniques. It is used to discover trends, and patterns, or to check assumptions with the help of statistical summaries and graphical representations. While performing the EDA of the urban Sound dataset we will analyze how the sound pattern varies at different locations of the city.

Feature Engineering: Feature Engineering helps to derive some valuable features from the existing ones. These extra features sometimes help in increasing the performance of the model significantly and certainly help to gain deeper insights into the data.

4. IMPLEMENTATION

4.1. LIBRARIES USED

NUMPY

NumPy (Numerical Python) is a fundamental library for scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently.

NumPy's core strength lies in its ability to perform rapid array operations, which are crucial for tasks like image processing, linear algebra, Fourier transforms, and statistical analysis.

PANDAS

Pandas is a Python library used for working with data sets.
It has functions for analyzing, cleaning, exploring, and manipulating data.
The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008.

SEABORN

Seaborn is a library for making statistical graphics in Python. It builds on top of matplotlib and integrates closely with pandas data structures.

MATPLOTLIB

Matplotlib is a comprehensive plotting library for Python, designed to create a wide variety of static, animated, and interactive visualizations.

The library supports various plot types including line plots, scatter plots, bar charts, histograms, 3D plots, and more. Its object-oriented interface enables the creation of publication-quality figures in various formats.

I/O (INPUT/OUTPUT) LIBRARY

The IO (Input/Output) library in Python is a versatile and essential component for handling various types of data streams. It provides a unified interface for working with text, binary, and raw I/O operations, offering consistent methods across different sources like files, network connections, and in-memory buffers. The IO library provides the tools necessary for efficient and reliable data handling in Python applications.

4.2. GRAPHS USED

Scatterplot

A scatter plot is a data visualization technique that displays individual data points on a two-dimensional graph. Each point on the plot represents the values of two variables, typically shown on the x and y axes. Scatter plots are excellent for revealing relationships, patterns, or correlations between variables.

They can quickly illustrate trends, clusters of data points, or outliers within a dataset.

Heatmap

A heatmap is a data visualization technique that represents numerical values as colors in a two-dimensional grid.

It's effective for displaying complex datasets and revealing patterns, trends, or correlations that might be difficult to discern from raw numbers alone.

In a heatmap, each cell's color intensity corresponds to its value, with warmer colors (like reds and oranges) typically representing higher values and cooler colors (like blues and greens) representing lower values.

This color-coding allows for quick visual assessment of data distribution and intensity across different categories or variables.

4.3. Example Tools and Technologies:

Data Collection: SONYC Urban Sound dataset (4 years)
Data Transformation: Python (Pandas, NumPy)
Exploratory Data Analysis (EDA): Python, seaborn.
Visualization: Graphs
Reporting: VS Code, Jupyter Notebooks.

5. PROJECT DEVELOPMENT

CODING

5.1. IMPORTING LIBRARIES

```
# importing libraries
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
```

IMPORTING DATA SET

Importing the Dataset

In this section, we will import the dataset from a file (CSV, Excel, etc.) and load it into our environment for analysis.

```
]: df= pd.read_csv('annotations.csv',low_memory=False)
   print("Read file successfully")
```

```
Read file successfully
```

5.2. READING DATA SET

Reading the Dataset

Now that we've imported the file, let's read the data and inspect its structure to understand its contents.

```
[3]: # Displaying basic information about the dataset
     df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 57449 entries, 0 to 57448
Data columns (total 78 columns):
 #   Column                                Non-Null Count  Dtype
---  ------                                --------------  -----
 0   split                                 57449 non-null  object
 1   sensor_id                             57449 non-null  int64
 2   audio_filename                        57449 non-null  object
 3   annotator_id                          57449 non-null  int64
 4   borough                               57449 non-null  int64
 5   block                                 57449 non-null  int64
 6   latitude                              57449 non-null  float64
 7   longitude                             57449 non-null  float64
 8   year                                  57449 non-null  int64
 9   week                                  57449 non-null  int64
 10  day                                   57449 non-null  int64
 11  hour                                  57449 non-null  int64
 12  1-1_small-sounding-engine_presence    57449 non-null  float64
```

Step 1: Data Understanding

```
[4]: #prints by default,initial 5 value
     df.head()
```

[4]:

	split	sensor_id	audio_filename	annotator_id	borough	block	latitude	longitude	year	week	...	7-X_other-unknown-human-voice_proximity	8-1_dog-barking-whining_proximity	1_engine_presence	2_machinery-impact_presence	3_no machiner impact_presen
0	test	0	00_026884.wav	0	1	547	40.72951	-73.99388	2019	43	...	-1	-1	-1	-1	
1	test	0	00_026919.wav	0	1	547	40.72951	-73.99388	2019	20	...	-1	-1	-1	-1	
2	test	0	00_027065.wav	0	1	547	40.72951	-73.99388	2019	35	...	-1	-1	-1	-1	
3	test	0	00_027096.wav	0	1	547	40.72951	-73.99388	2019	31	...	-1	-1	-1	-1	
4	test	0	00_027130.wav	0	1	547	40.72951	-73.99388	2019	40	...	-1	-1	-1	-1	

5 rows × 78 columns

we are printing the first 5 values of data set in above output

we are printing the first 5 values of data set in above output

[5]:
```
#print, last 5 values by default
df.tail()
```

[5]:

	split	sensor_id	audio_filename	annotator_id	borough	block	latitude	longitude	year	week	...	7-X_other-unknown-human-voice_proximity	8-1_dog-barking-whining_proximity	1_engine_presence	2_machinery-impact_presence	m impact
57444	validate	46	46_020807.wav	5399	1	559	40.73365	-73.98879	2018	42	...	-1	-1	0	0	
57445	validate	46	46_020807.wav	5424	1	559	40.73365	-73.98879	2018	42	...	-1	-1	0	0	
57446	validate	46	46_020853.wav	5337	1	559	40.73365	-73.98879	2018	34	...	far	-1	0	0	
57447	validate	46	46_020853.wav	5365	1	559	40.73365	-73.98879	2018	34	...	-1	-1	0	0	
57448	validate	46	46_020853.wav	5373	1	559	40.73365	-73.98879	2018	34	...	-1	-1	1	0	

5 rows × 78 columns

we are printing the last 5 values of data set in above output

```
#Checking for missing value

print("Missing value count in each column- \n",df.isnull().sum())

# isnull() will identify if there is any null value indataset or not(returns the result in true/false)
# sum() will identify total count of true for isnull()
```

```
Missing value count in each column-
 split                        0
sensor_id                     0
audio_filename                0
annotator_id                  0
borough                       0
                             ..
4_powered-saw_presence        0
5_alert-signal_presence       0
6_music_presence              0
7_human-voice_presence        0
8_dog_presence                0
Length: 78, dtype: int64
```

There were no null values present in the dataset

```
# checking the shape the dataset
df.shape
```

```
(57449, 78)
```

9]:
```
 # Checks the data type of each columns
df.dtypes
```

9]:
```
split                      object
sensor_id                   int64
audio_filename             object
annotator_id                int64
borough                     int64
                            ...
4_powered-saw_presence      int64
5_alert-signal_presence     int64
6_music_presence            int64
7_human-voice_presence      int64
8_dog_presence              int64
Length: 78, dtype: object
```

The above function gives the datatype of a particular columns is haning in the dataset

5.3. EXPLORATORY DATA ANALYSIS

Data Visualization (EDA)

In this section, we will visualize the data to gain insights and better understand the relationships between different variables.

1.--- LINE PLOTS ---

```
# Group by hour and count car horn presence
car_horn_hourly = df.groupby('hour')['5-1_car-horn_presence'].sum()

# Plot the line chart
plt.figure(figsize=(10, 5))
plt.plot(car_horn_hourly.index, car_horn_hourly.values, marker='o', linestyle='-', color='tomato')
plt.title("Hourly Car Horn Frequency")
plt.xlabel("Hour of Day")
plt.ylabel("Number of Car Horn Events")
plt.grid(True)
plt.xticks(range(0, 24))
plt.tight_layout()
plt.show()
```

```
# Define the noise tags to analyze
tags_to_plot = [
    '5-1_car-horn_presence',
    '5-3_siren_presence',
    '6-1_stationary-music_presence',
    '7-1_person-or-small-group-talking_presence'
]

# Group by hour and sum each tag's presence
tag_hourly_data = df.groupby('hour')[tags_to_plot].sum()
print(tag_hourly_data )
```

```
# Plotting the grapg
plt.figure(figsize=(12, 6))
for tag in tags_to_plot:
    # Clean up label names for legend
    label = tag.replace('_presence', '').replace('-', ' ').replace('5 1', 'Car Horn').replace('5 3', 'Siren').replace('6 1', 'Music').replace('7 1', 'Voice')
    plt.plot(tag_hourly_data.index, tag_hourly_data[tag], marker='o', label=label)

plt.title("Hourly Trends of Urban Noise Events")
plt.xlabel("Hour of Day")
plt.ylabel("Number of Events Detected")
plt.xticks(range(0, 24))
plt.legend(title="Noise Type")
plt.grid(True)
plt.tight_layout()
plt.show()
```

2.--- BAR PLOTS ---

```
#mapping borough with its name
df['borough_name'] = df['borough'].map({
    1: 'Manhattan', 2: 'Bronx', 3: 'Brooklyn', 4: 'Queens', 5: 'Staten Island'
})

# Count number of audio files in each borough
borough_counts = df['borough_name'].value_counts()

# Plot the bar chart
plt.figure(figsize=(6, 4))
borough_counts.plot(kind='bar', color='skyblue')
plt.title("Audio Files per Borough")
plt.ylabel("Count")
plt.xlabel("Borough")
plt.grid(True)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

3.--- COUNT PLOTS ---

```
# Plot count of audio files by borough
plt.figure(figsize=(8, 5))
sns.countplot(data=df, x='borough_name', palette='viridis')
plt.title("Audio File Count by Borough")
plt.xlabel("Borough")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
# Plot count of rows for each split type (train, test, val)

plt.figure(figsize=(6, 4))
sns.countplot(data=df, x='split', palette='pastel')
plt.title("Data Split Distribution")
plt.xlabel("Split Type")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

3.--- BOX PLOTS ---

```
# Plot car horn presence by borough
plt.figure(figsize=(8, 5))
sns.boxplot(x='borough_name', y='5-1_car-horn_presence', data=df, palette='Set3')
plt.title("Car Horn Presence by Borough")
plt.xlabel("Borough")
plt.ylabel("Presence Value")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
# Select columns related to sound category presence (indices 70 to 77)
presence_columns = df.columns[70:78]

# Create box plot
plt.figure(figsize=(10, 6))
sns.boxplot(data=df[presence_columns])
plt.title("Box Plot of Sound Category Presence")
plt.xticks(rotation=45)
plt.ylabel("Presence Value")
plt.tight_layout()
plt.show()
```

4.--- SCATTER PLOTS ---

```
# Scatter Plot 1: Car horn presence vs. year
plt.figure(figsize=(10, 5))
sns.scatterplot(x='year', y='5-1_car-horn_presence', data=df, alpha=0.3)
plt.title("Car Horn Presence vs. Year")
plt.ylabel("Car Horn Presence")
plt.xlabel("Year")
plt.grid(True)
plt.show()
```

```
# Filter out invalid engine presence values (-1 likely means missing)
df_filtered = df[df["1_engine_presence"] != -1]

# Create the scatter plot
plt.figure(figsize=(10, 5))
sns.scatterplot(data=df_filtered, x="hour", y="1_engine_presence", hue="borough_name")
plt.title("Engine Presence by Hour and Borough")
plt.xlabel("Hour of Day")
plt.ylabel("Engine Presence")
plt.legend(title="Borough")
plt.grid(True)
plt.tight_layout()
plt.show()
```

5.--- HISTOGRAMS PLOTS ---

```
# Converting '2-1_rock-drill_proximity' to numeric
df['2-1_rock-drill_proximity'] = pd.to_numeric(df['2-1_rock-drill_proximity'], errors='coerce')

# Plotting the histogram
plt.figure(figsize=(6, 4))
plt.hist(df['2-1_rock-drill_proximity'].dropna(), bins=20, color='purple', edgecolor='black')
plt.title("Rock Drill Proximity")
plt.xlabel("Proximity")
plt.ylabel("Count")
plt.grid(True)
plt.show()
```

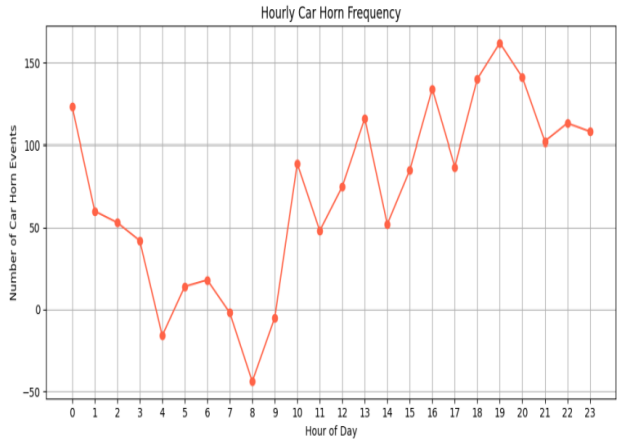
```
# Converting '2-1_rock-drill_proximity' to numeric
df['4-1_chainsaw_proximity'] = pd.to_numeric(df['4-1_chainsaw_proximity'], errors='coerce')

# Plotting the histogram
plt.figure(figsize=(6, 4))
plt.hist(df['4-1_chainsaw_proximity'].dropna(), bins=20, color='green', edgecolor='black')
plt.title("Chainsaw Proximity")
plt.xlabel("Proximity")
plt.ylabel("Count")
plt.grid(True)
plt.show()
```

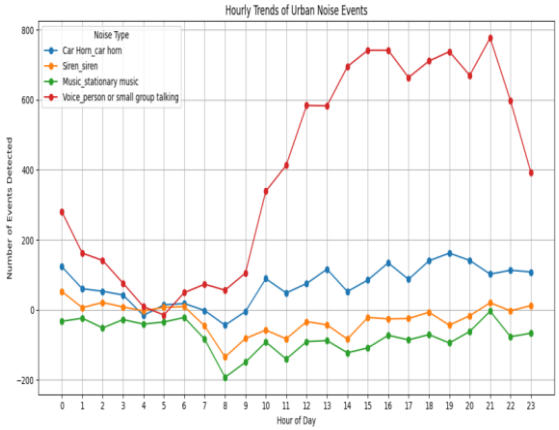
6.--- PIE PLOTS ---

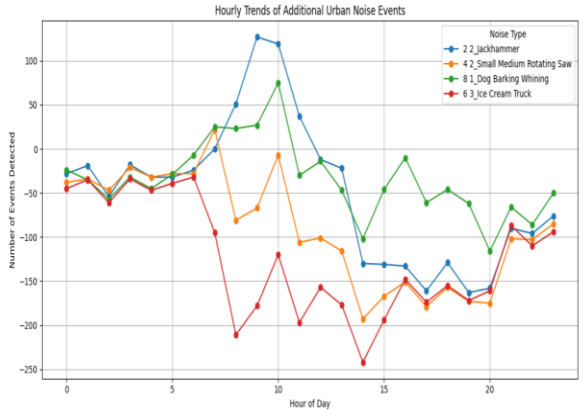
```
# Plotting the graph

plt.figure(figsize=(6, 6))
plt.pie(borough_counts, labels=borough_counts.index, autopct='%1.1f%%', startangle=140,colors=['#7ec8e3','#c26dbc','#c8f4f9','#3cacae'])
plt.title("Distribution of Recordings by Borough")
plt.axis('equal')
plt.show()
```

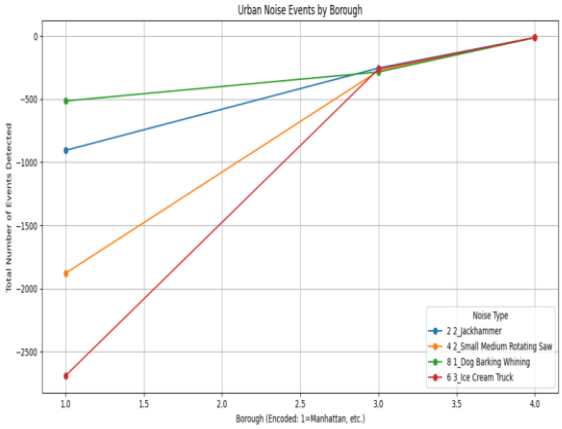
```
# Count by car horn presence
presence_counts = df["1-3_large-sounding-engine_presence"].value_counts()

# Map values to readable labels
labels = presence_counts.index.map({-1: "Unknown", 0: "Absent", 1: "Present"})

# Plot
plt.figure(figsize=(6, 6))
plt.pie(presence_counts, labels=labels, autopct='%1.1f%%',colors=['#f4ebd0','#36eee0'])
plt.title("Engine sound Presence Distribution")
plt.axis('equal')
plt.show()
```

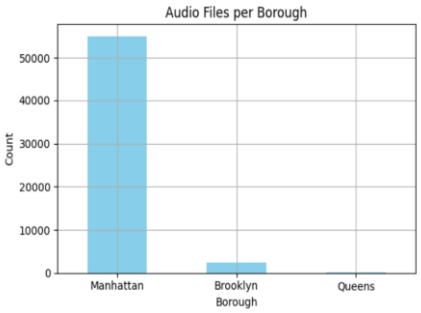
Exploratory data analysis is an approach to analyzing the data using visual techniques. It is used to discover trends, and patterns, or to check assumptions with the help of statistical summaries and graphical representations.

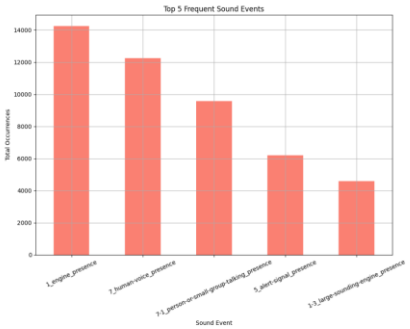


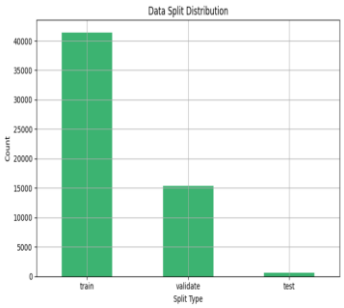


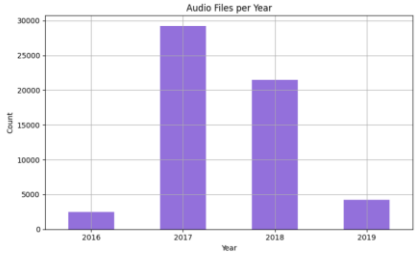


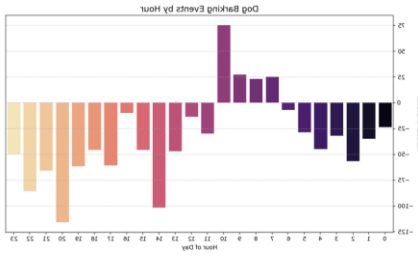


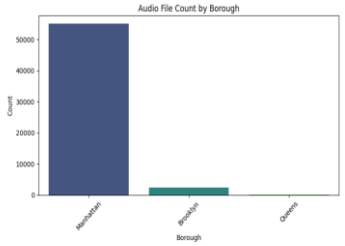

Audio Files per Borough
Count
Manhattan
Brooklyn
Queens
Borough

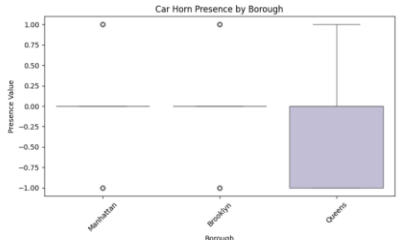

Top 5 Frequent Sound Events
Total Occurrences
1_engine_presence
7_human-voice_presence
Sound Event

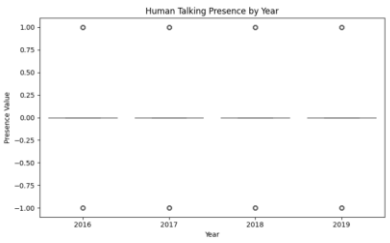

Data Split Distribution
Count
train
validate
test
Split Type

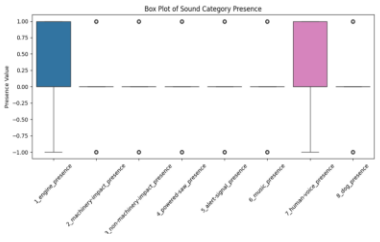

Audio Files per Year
Count
2016
2017
2018
2019
Year

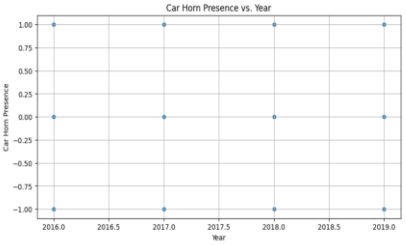

Dog Barking Events by Hour
Hour of Day

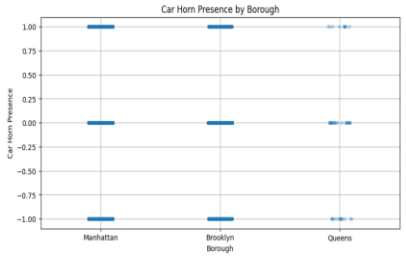

Audio File Count by Borough
Count
Manhattan
Brooklyn
Queens
Borough

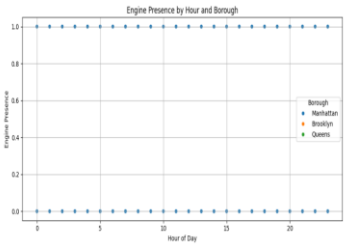

Car Horn Presence by Borough
Presence Value
Manhattan
Brooklyn
Queens

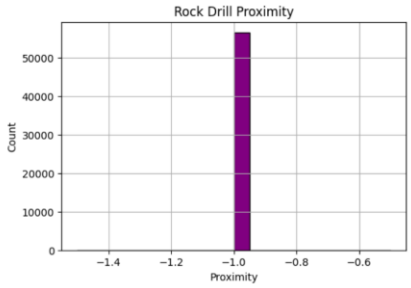

Human Talking Presence by Year
Presence Value
2016
2017
2018
2019
Year

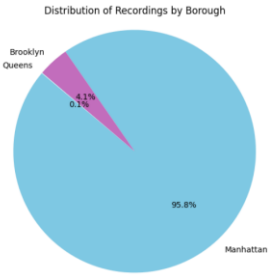

Box Plot of Sound Category Presence
Presence Value
1_engine_presence
7_human-voice_presence
8_dog_presence

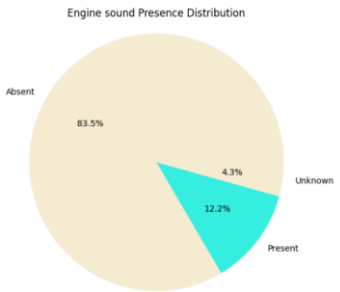

Car Horn Presence vs. Year
Car Horn Presence
Year


Car Horn Presence by Borough
Car Horn Presence
Manhattan
Brooklyn
Queens
Borough


Engine Presence by Hour and Borough
Engine Presence
Hour of Day
Borough
Manhattan
Brooklyn
Queens


Rock Drill Proximity
Count
Proximity


Distribution of Recordings by Borough
Brooklyn
Queens
4.1%
0.1%
95.8%
Manhattan

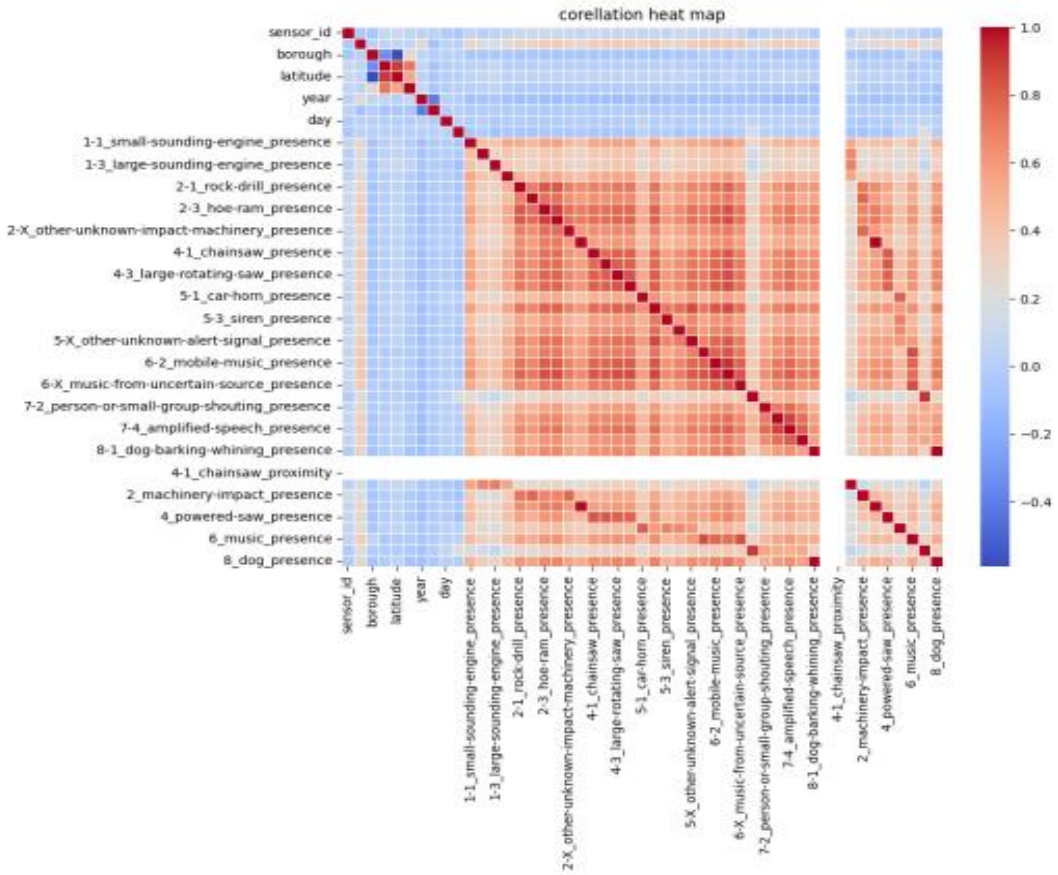

Engine sound Presence Distribution
Absent
83.5%
4.3%
Unknown
12.2%
Present

The visual analysis revealed that engine sounds, sirens, and human voices dominate urban noise, especially during daytime working hours. Spatial plots showed high noise density in central boroughs like Manhattan. Pie charts confirmed engines as the most frequent noise source, while the heatmap showed strong co-occurrence between traffic and alert sounds. These patterns highlight how urban activity types contribute to city noise and can support better planning and noise regulation**.**

7.--- CORRELATION MATRIX USING HEAT MAP ---

]:
```
#selecting num col
num_col=df.select_dtypes(include=['int64','float64',])

 # It is used to create a correlation matrix
cor_mat=num_col.corr()
```

]:
```
# visualising correlation matrix using using HeatMap

plt.figure(figsize=(10,8))
sns.heatmap(cor_mat,cmap='coolwarm',linewidths=0.5)
plt.title('corellation heat map')
```



From the above Heat map we observe that :-

- Many sound events (like engine, horn, siren) are positively correlated, often occurring together.
- Location and time features (latitude, year, etc.) show very weak correlation with sound events.
- Proximity values (like chainsaw proximity) correlate strongly with their corresponding presence features.
- Sound types form natural clusters (e.g., music sounds grouped together).

6. CONCLUSION

This project effectively demonstrates how mobile-based citizen-contributed recordings can be leveraged to analyze urban noise patterns in a scalable and insightful manner. By examining a large dataset of annotated sound clips collected from various boroughs and blocks, we uncovered meaningful trends across both space (geolocation) and time (year, week, day, and hour).
Through exploratory data analysis and visualization:

- We identified the prevalence of noise types such as engine sounds, sirens, human voices, and construction tools in different areas.
- Temporal patterns revealed peak noise levels during working hours and weekdays, especially in densely populated regions.
- The heatmaps and correlation matrices highlighted relationships between various sound events, such as co-occurrence of sirens and large crowds or barking dogs and residential zones.

These insights can support urban planning, noise regulation, and public health strategies by pinpointing problem areas and understanding behavioral trends. The study showcases the potential of participatory sensing combined with data science to drive smart, citizen-aware urban management.

7. REFERENCES

1. Cartwright, M., Cramer, J., Salamon, J., Bello, J.P. (2020). *SONYC Urban Sound Dataset*. New York University. Retrieved from: https://wp.nyu.edu/sonyc/
2. Wes McKinney. (2010). *Data Structures for Statistical Computing in Python*. In Proceedings of the 9th Python in Science Conference, pp. 51–56.
3. Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3), 90-95.
4. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
5. Seaborn Documentation. *Statistical Data Visualization in Python*. Retrieved from: https://seaborn.pydata.org
6. OpenStreetMap. *Location Mapping Data (Latitude/Longitude)*. Retrieved from: https://www.openstreetmap.org